Scaling to 800 Scenes — Case 1: Retraining UNet_Night

Introduction

The dataset generation finished with 679 fully verified scenes (509 train / 170 test) stored as compressed arrays in `D:\FINDFIRE_DATA\processed_800\`.

To keep our progression controlled and transparent, we will handle the pipeline in three distinct, sequential cases:

```
SCALING WORKFLOW (ONE BY ONE)

CASE 1: Retrain UNet_Night on the 800-dataset (Pure Thermal 3-Ch)
CASE 2: Retrain UNet_Day on the 800-dataset (11x11 Contrast 4-Ch)
CASE 3: Benchmark both models on the 170 untouched test scenes
```

We begin strictly with **CASE 1: The Night Model (UNet_Night)**.

Understanding Case 1: Night Model Retraining

Before running the code, let's establish the mechanics:

- **Input Representation:** 3 physical channels $[T_{3.9}, T_{10.8}, \Delta T]$ padded to $[128, 256]$.
- **Data Selection:** We query `dataset_index_800.csv` strictly for nocturnal passes (UTC Hour < 6 or UTC Hour > 18).
- **Loss Function:** Focal Tversky Loss with $\alpha = 0.3$, $\beta = 0.7$, $\gamma = 4/3$. Because the night sky is free of solar glint, $\beta = 0.7$ directs the optimizer to detect faint, isolated fire fronts.
- **Checkpoint Isolation:** The weights will be saved to `checkpoints/best_unet_night_800.pth` so your previous working checkpoints remain untouched.

Step 1: Create `train_night_800.py`

Create `train_night_800.py` in `D:\ramy uni\AI ENGENEERING\FINDFIRE\findfire\`. This script contains its own self-contained dataset loader to ensure no 3-channel vs. 4-channel conflicts occur:

```python
import os
import time
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.utils.data import DataLoader, Dataset

from unet_model import UNet

#
    ================================================================

# 1. NIGHT DATASET LOADER (STRICTLY 3 CHANNELS)
#
    ================================================================

class WildfireNightDataset800(Dataset):
  def __init__(self, index_csv_path, split="train", augment=False
      ):
    self.split = split
    self.augment = augment

    df = pd.read_csv(index_csv_path)
    df["hour"] = pd.to_datetime(df["timestamp"]).dt.hour

    # Filter strictly for nocturnal scenes (night regime)
    night_mask = (df["hour"] < 6) | (df["hour"] > 18)
    split_mask = df["split"] == split
    self.records = df[night_mask & split_mask].reset_index(drop=
        True)

    if len(self.records) == 0:
      raise RuntimeError(f"No night scenes found for split='{
          split}'")

    # Thermal channel normalization (in Kelvin)
    self.means = torch.tensor([295.0, 285.0, 10.0]).view(3, 1, 1)
    self.stds = torch.tensor([15.0, 12.0, 8.0]).view(3, 1, 1)

  def __len__(self):
    return len(self.records)

  def __getitem__(self, idx):
    row = self.records.iloc[idx]
    data = np.load(row["file_path"])
```

```
    image = torch.from_numpy(data["image"].astype(np.float32))  # [3, 120, 250]
    mask = torch.from_numpy(data["mask"].astype(np.float32)).unsqueeze(0)

    # Standardize
    image = (image - self.means) / self.stds

    # Pad (left=3, right=3, top=4, bottom=4) -> [3, 128, 256]
    image_padded = F.pad(image, (3, 3, 4, 4), mode="reflect")
    mask_padded = F.pad(mask, (3, 3, 4, 4), mode="constant", value=0.0)

    if self.augment:
      if torch.rand(1).item() > 0.5:
        image_padded = torch.flip(image_padded, dims=[2])
        mask_padded = torch.flip(mask_padded, dims=[2])
      if torch.rand(1).item() > 0.5:
        image_padded = torch.flip(image_padded, dims=[1])
        mask_padded = torch.flip(mask_padded, dims=[1])

    return image_padded, mask_padded


#
    ================================================================

# 2. FOCAL TVERSKY LOSS
#
    ================================================================

class FocalTverskyLoss(nn.Module):
  def __init__(self, alpha=0.3, beta=0.7, gamma=4.0 / 3.0, eps=1e-6):
    super(FocalTverskyLoss, self).__init__()
    self.alpha = alpha
    self.beta = beta
    self.gamma = gamma
    self.eps = eps

  def forward(self, logits, targets):
    probs = torch.sigmoid(logits)
    p_flat = probs.view(probs.size(0), -1)
    t_flat = targets.view(targets.size(0), -1)

    tp = torch.sum(p_flat * t_flat, dim=1)
    fp = torch.sum(p_flat * (1.0 - t_flat), dim=1)
    fn = torch.sum((1.0 - p_flat) * t_flat, dim=1)

    ti = (tp + self.eps) / (tp + self.alpha * fp + self.beta * fn + self.eps)
```

```
        loss = torch.pow((1.0 - ti), (1.0 / self.gamma))
        return torch.mean(loss)


#
    ================================================================

# 3. CONFIGURATION & TRAINING LOOP
#
    ================================================================

INDEX_PATH = r"D:\FINDFIRE_DATA\processed_800\dataset_index_800.
    csv"
CHECKPOINT_DIR = "checkpoints"
os.makedirs(CHECKPOINT_DIR, exist_ok=True)

BATCH_SIZE = 8
EPOCHS = 40
LEARNING_RATE = 2e-4
WEIGHT_DECAY = 1e-4

device = torch.device("cuda" if torch.cuda.is_available() else "
    cpu")
print("====================================================")
print(f"TRAINING UNET_NIGHT (800 DATASET) ON: {device}")
if device.type == "cuda":
  print(f"GPU: {torch.cuda.get_device_name(0)}")
print("====================================================")

train_dataset = WildfireNightDataset800(INDEX_PATH, split="train"
    , augment=True)
val_dataset = WildfireNightDataset800(INDEX_PATH, split="test",
    augment=False)

train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True,
        num_workers=0
)
val_loader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False,
        num_workers=0
)

print(f"Nighttime Training Scenes   : {len(train_dataset)} ({len(
    train_loader)} batches)")
print(f"Nighttime Validation Scenes : {len(val_dataset)} ({len(
    val_loader)} batches)\n")

model = UNet(in_channels=3, out_channels=1, base_features=64).to(
    device)
criterion = FocalTverskyLoss(alpha=0.3, beta=0.7, gamma=4.0 /
```

```
    3.0)
optimizer = AdamW(model.parameters(), lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY)
scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS, eta_min=1e
    -6)

best_val_dice = 0.0
best_model_path = os.path.join(CHECKPOINT_DIR, "
    best_unet_night_800.pth")

print("Beginning Case 1 Training (UNet_Night)...")
start_time = time.time()

for epoch in range(1, EPOCHS + 1):
  epoch_start = time.time()
  model.train()
  train_loss = 0.0

  for images, masks in train_loader:
    images = images.to(device)
    masks = masks.to(device)

    logits = model(images)
    loss = criterion(logits, masks)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    train_loss += loss.item()

  train_epoch_loss = train_loss / len(train_loader)

  # Validation sweep at threshold 0.50
  model.eval()
  val_loss = 0.0
  val_dice_list, val_recall_list = [], []

  with torch.no_grad():
    for images, masks in val_loader:
      images = images.to(device)
      masks = masks.to(device)

      logits = model(images)
      loss = criterion(logits, masks)
      val_loss += loss.item()

      probs = torch.sigmoid(logits)
      preds = (probs >= 0.50).float()

      p_flat = preds.view(-1)
```

```
        t_flat = masks.view(-1)
        tp = torch.sum(p_flat * t_flat).item()
        fp = torch.sum(p_flat * (1.0 - t_flat)).item()
        fn = torch.sum((1.0 - p_flat) * t_flat).item()

        dice = (2.0 * tp + 1e-6) / (2.0 * tp + fp + fn + 1e-6)
        recall = (tp + 1e-6) / (tp + fn + 1e-6)
        val_dice_list.append(dice)
        val_recall_list.append(recall)

    val_epoch_loss = val_loss / len(val_loader)
    mean_val_dice = np.mean(val_dice_list)
    mean_val_recall = np.mean(val_recall_list)

    scheduler.step()
    lr = scheduler.get_last_lr()[0]
    duration = time.time() - epoch_start

    saved_flag = ""
    if mean_val_dice > best_val_dice:
      best_val_dice = mean_val_dice
      torch.save(
          {
              "epoch": epoch,
              "model_state_dict": model.state_dict(),
              "best_dice": best_val_dice,
              "val_loss": val_epoch_loss,
          },
          best_model_path,
      )
      saved_flag = "--> [SAVED BEST NIGHT MODEL]"

    print(
        f"Epoch [{epoch:02d}/{EPOCHS:02d}] ({duration:.1f}s) | LR:
           {lr:.6f} | "
        f"Train Loss: {train_epoch_loss:.4f} | Val Loss: {
           val_epoch_loss:.4f} | "
        f"Val Dice: {mean_val_dice:.4f} | Val Recall: {
           mean_val_recall:.4f} {saved_flag}"
    )

total_mins = (time.time() - start_time) / 60
print("\n==================================================")
print("CASE 1 COMPLETE: UNet_Night Training Finished")
print(f"Elapsed Time     : {total_mins:.2f} minutes")
print(f"Best Val Dice    : {best_val_dice:.4f}")
print(f"Saved Checkpoint : {best_model_path}")
print("==================================================")
```

Step 2: Run Case 1

Execute the training script:

```
python train_night_800.py
```

Let the 40 epochs complete on your GTX 1660 SUPER. Once it finishes and saves `checkpoints/best_unet_night_800.pth`, post the final summary lines, and we will proceed to Case 2.